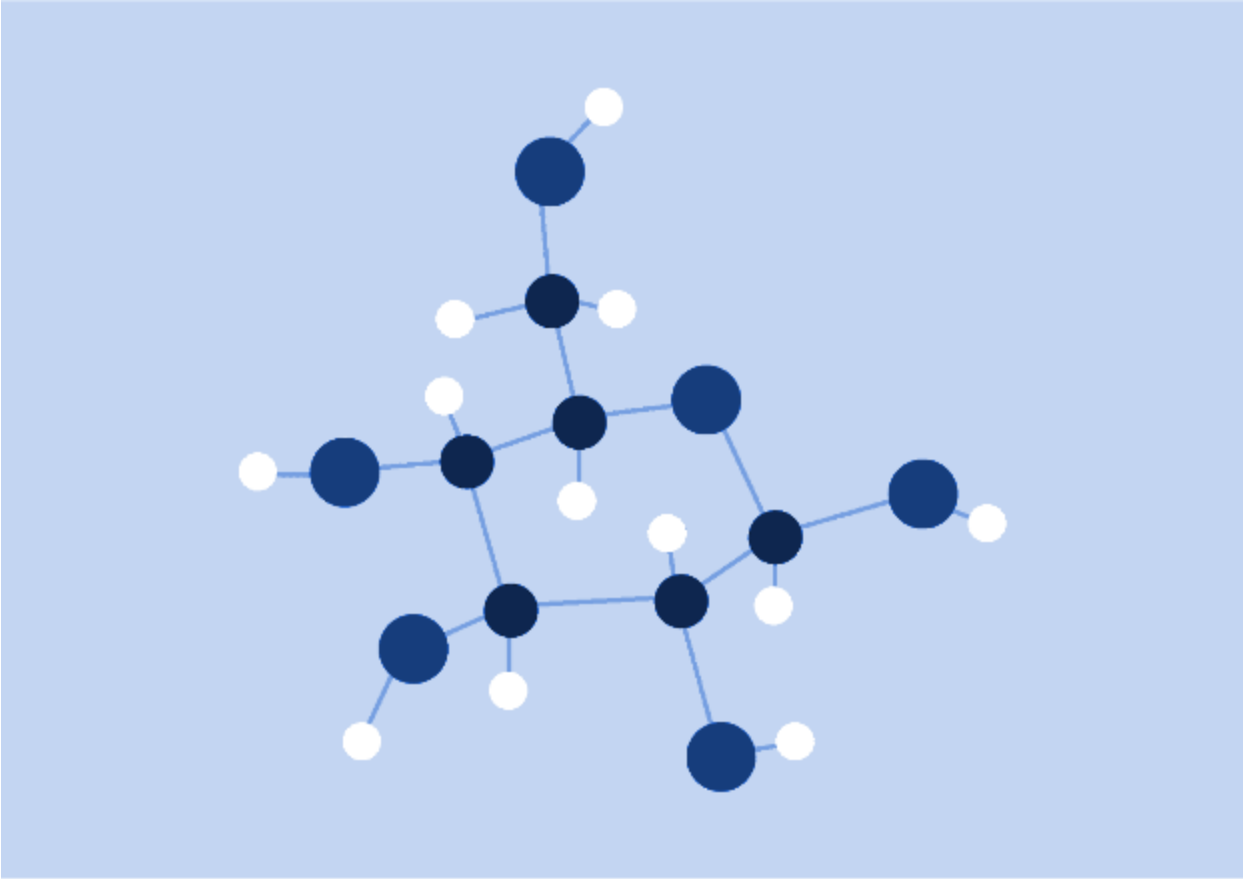
Style- and Documentation Guide



NI0 - Name Ideas 0

24.03.2026

Author: Konstantin Fastovski

Proof reader: Paul Berndt

Outline

Outline **1**
Version Control **2**
Naming: 2
Pull Request Policy: 3
Code **3**
Naming: 3
Tools: 4
Frontend Documentation: **5**
TSDoc: 5
README: 5
Inline Comments: 5
Backend Documentation: **5**
Javadoc: 5
API Documentation: 6
Design Documentation: **6**
Feature & Issue Documentation **6**
Sources **7**

Version Control

Naming:

Component	Style	Example	Note
Commit messages	In English, imperative; describe your commit as the ending of the sentence "This commit will..." Optionally add a tag like "feat:"	add user account handling feat: initialize spring project fix: prevent crash on empty password	If you have to use "and" in your commit message, you should split your changes into two commits. (Your future self will thank you.)
Branches	Pattern for branch names: <type>/<descriptive-name> Branch types: feature/ – New features fix/ – Bug fixes chore/ – Maintenance tasks (e.g., updating packages) refactor/ – Code refactoring without behavioral changes doc/ – docs test/ – tests	fix/exception-thrown-on-login chore/update-angular	The main branch should always contain a working product!

Pull Request Policy:

Documentation changes that accompany a code change must be included in the same PR. A PR must not be merged if the documentation does not meet the standards set in Frontend Documentation and Backend Documentation.

Code

Naming:

Component	Style	Example	Note
Files	kebab-case	component.ts, example-component.spec.ts	Matches the name of the primary nameable identifier, unless there are multiple, in which case a name that matches the general theme of the code within the file is chosen (not too general!).
Components/ Classes	PascalCase	UserProfile	
Temporary Variables	snake_case	i, last_name	
Object Properties	camelCase	userName	

Tools:

Usecase	Language	Tool	Usage
Linting	TypeScript	typescript-eslint	The recommended settings are used and enforced before pushing to the main branch.
	Java	SonarlLint, Checkstyle	The Google checks are enforced before pushing to the main branch.
	CSS	ESLint	The recommended settings are used and enforced before pushing to the main branch.
	HTML	ESLint	The recommended settings are used and enforced before pushing to the main branch.
Formatting	Typescript	Prettier(d)	Standard config
	Java	Prettier(d)	Standard config
	CSS	Prettier(d)	Standard config
	HTML	Prettier(d)	Standard config

Frontend Documentation:

The responsibility for the documentation lies with whoever pushes the change.

TSDoc:

All exported Angular classes, components, services, pipes and directives must carry a TSDoc block comment. The comment must include a summary, `@param` and `@returns` tags where applicable and an `@example` snippet for non-trivial usage.

README:

Every standalone feature-module directory must contain a `README.md` covering:

1. Purpose and responsibility of the module or component.
2. Input / Output bindings with type and description.
3. Dependencies on services or third-party libraries.
4. Known limitations or edge cases.

Inline Comments:

- Comments must explain why — not what. The code itself explains what.
- TODO and FIXME comments must include an issue reference. (`// TODO \[APOLLO-42\]: do something`)
- Commented-out code must not be committed to the main branch.
- Magic numbers and complex regular expressions must have an adjacent explanatory comment.

Backend Documentation:

The responsibility for the documentation lies with whoever pushes the change.

Javadoc:

All public classes, interfaces, enumerations and public/protected methods must have a Javadoc block comment. Package-level documentation must be provided in `package-info.java` files.

Every custom exception class must carry a Javadoc comment that describes:

- The error condition that causes it to be thrown.
- Recovery or retry advice for API consumers.

API Documentation:

The REST API must be fully described in an OpenAPI 3.1 specification file (e.g., `/docs/api/openapi.yml` in backend repository). The specification must be generated from code annotations (Springdoc OpenAPI) rather than maintained by hand.

Element	Requirement
Info block	Must include title, version, contact email and license.
Every endpoint	Must have a summary (<= 10 words) and a description.
Responses	Must document all HTTP status codes including 4xx and 5xx.
Deprecated options	Must use `deprecated: true` flag plus a successor link.

Design Documentation:

The single source of truth for the current architectural design is kept in the files `frontend.drawio`, `frontend.md`, `backend.drawio` and `backend.md` in their respective repositories. Changes to these documents are made with the knowledge and consent of all team members. The responsibility for enforcing the decisions made lies with the chief of modeling, Felix Weglehner.

Feature & Issue Documentation

Planned features, user stories, and bug reports are kept in Jira. Pull requests that merge the changes suggested in these issues must contain a link to the corresponding issue.

Sources

[1] Software documentation guide — Write the Docs